Общее описание системы:

Сервис предназначен для онлайн-бронирования рабочих мест в сети «CoBooking».

Пользователи могут находить рабочие пространства по различным параметрам, просматривать подробную информацию, бронировать места, оплачивать бронирования и управлять ими через личный кабинет.

Система обеспечивает владельцам пространств инструменты для управления доступностью мест, мониторинга загрузки и автоматизации операций.

Цели системы:

Повысить удобство и скорость бронирования рабочих мест для пользователей.

Обеспечить цифровую платформу для масштабирования сети рабочих пространств.

Увеличить загрузку рабочих мест и общую прибыльность бизнеса.

Снизить операционные издержки за счёт автоматизации процессов бронирования и оплаты.

Создать единую точку доступа к информации о доступных рабочих пространствах.

Функциональные требования:

Управление пользователями:

- Регистрация и вход по email и паролю.
- Аутентификация через внешние провайдеры (Google, VK, Яндекс).
- Управление профилем пользователя.

Просмотр и редактирование личных данных:

- Поиск и выбор рабочих мест
- Поиск рабочих мест по городу.
- Отображение рабочих пространств на карте.
- Фильтрация по доступности, расстоянию и параметрам.
- Просмотр детальной информации о рабочем месте.

Бронирование:

Создание бронирования на выбранное время.

Просмотр деталей бронирования.

Отмена бронирования.

Изменение параметров бронирования (при допустимых условиях).

Оплата:

Онлайн-оплата банковской картой

Оплата через СБП.

Выбор оплаты на месте.

Обработка статусов оплаты.

Повторная попытка оплаты при неудаче.

Личный кабинет:

Просмотр текущих и завершённых бронирований.

История оплат:

Управление избранными рабочими местами.

Просмотр статусов бронирований.

Уведомления:

Уведомление о подтверждении бронирования.

Напоминания о предстоящем бронировании.

Уведомления об изменении статуса бронирования.

Нефункциональные требования:

Безопасность

Защита персональных данных пользователей.

Безопасное хранение паролей и токенов.

Использование защищённых протоколов передачи данных.

Контроль доступа к данным и операциям.

Производительность:

Время отклика поиска и загрузки результатов не более нескольких секунд.

Обработка одновременных запросов без заметной деградации производительности.

Быстрое обновление статусов доступности рабочих мест.

Масштабируемость:

Возможность горизонтального масштабирования при росте нагрузки.

Поддержка увеличения количества пользователей и рабочих пространств.

Возможность расширения системы без остановки сервиса.

Доступность и надёжность

Высокая доступность сервиса.

Сохранение целостности данных при сбоях.

Восстанавливаемость после отказов.

Стек технологий:

Backend и API:

Go — основной язык серверной разработки (высокая производительность, простота масштабирования, эффективная работа с конкурентностью)

REST API — взаимодействие клиентских приложений и внешних интеграций

WebSocket — обновление статусов и доступности рабочих мест в реальном времени

Формат данных:

JSON — универсальный формат обмена данными между клиентом и сервером

Аутентификация и авторизация:

OAuth 2.0 / OIDC — поддержка входа через внешних провайдеров

JWT — авторизация запросов и управление сессиями пользователей

База данных и хранение данных:

PostgreSQL — основная реляционная база данных

PostGIS — хранение и обработка геоданных, поиск по расстоянию и координатам

Кэширование и производительность:

Redis — кэширование часто запрашиваемых данных, хранение сессий и rate limiting

Наблюдаемость и мониторинг:

Prometheus + Grafana — сбор и визуализация технических метрик системы (CPU, memory, latency, количество запросов, error rate).

Централизованное логирование (Loki / ELK) — сбор, хранение и анализ логов микросервисов, API Gateway и инфраструктуры.

Трассировка запросов (distributed tracing) — отслеживание цепочки запросов между микросервисами для диагностики задержек и ошибок.

Alerting — автоматическое оповещение при отклонении метрик от нормальных значений. Используются правила алертов для:

высокого времени ответа API

роста количества ошибок (5xx)

падения сервисов

проблем с подключением к базе данных

Health checks — регулярная проверка состояния сервисов через эндпоинты /health и /ready. Используется Kubernetes для автоматического перезапуска контейнеров при сбоях.

SLO и SLA метрики — отслеживание ключевых показателей доступности системы:
доступность API
время ответа сервисов
доля успешных запросов

Мониторинг бизнес-метрик — сбор продуктовых показателей системы:
конверсия поиска в бронирование
конверсия бронирования в оплату
средний чек бронирования
загрузка рабочих мест

Метрики собираются из событий системы и отображаются на аналитических дашбордах.

Хранение медиа и файлов:
Object Storage (S3-совместимое хранилище) — хранение изображений рабочих пространств и пользовательского контента

Асинхронное взаимодействие:
RabbitMQ — брокер сообщений для обмена событиями между сервисами и обработки фоновых задач

Consistency и целостность данных:
Транзакции PostgreSQL
Уникальные ограничения и индексы
Оптимистичные блокировки
Защита от двойного бронирования

Инфраструктура и деплой:
Docker — контейнеризация сервисов
Kubernetes — оркестрация и горизонтальное масштабирование
Nginx — маршрутизация запросов и балансировка нагрузки

Security Audit Logging
Система ведет аудит-лог для чувствительных операций.
Аудит-лог фиксирует следующие события:

удаление аккаунта пользователя
изменение email или пароля
инициирование возврата средств

отмена платежа

изменение роли пользователя

вход администратора

неудачные попытки авторизации

Каждая запись аудита содержит:

eventId

eventType

userId

actorId

ipAddress

userAgent

timestamp

metadata

Audit-лог хранится отдельно от аналитических событий
и не может изменяться пользователями.

Безопасность API:

HTTPS/TLS — защита передачи данных

Rate limiting — защита от злоупотреблений

Защита от brute force атак

CORS политика доступа

Шифрование чувствительных данных

Политика паролей:

Минимальная длина: 8 символов

Максимальная длина: 128 символов

Пароль должен содержать минимум:

одну заглавную букву

одну строчную букву

одну цифру

один специальный символ

Запрещено использовать:

email пользователя

часто используемые пароли (password, 123456 и др.)

повторение предыдущих паролей (последние 3)

Пароли хранятся только в виде хэша.

Алгоритм хэширования:

Argon2id / bcrypt

Salt генерируется автоматически.

CI/CD и управление релизами:

CI/CD pipeline — автоматическая сборка, тестирование и деплой

Версионирование API — поддержка совместимости клиентов

Документация API:

OpenAPI (Swagger) — документирование и тестирование API

Тестирование:

Система покрывается несколькими уровнями тестирования: unit-тестами, интеграционными тестами, API-тестами и нагрузочными тестами. Для каждого уровня определены конкретные сценарии.

1. Unit тестирование

Unit-тесты проверяют отдельные функции бизнес-логики внутри микросервисов без взаимодействия с внешними сервисами.

Примеры тестируемых сценариев:

- Создание бронирования
- проверяется:
  - корректность проверки времени бронирования
  - запрет создания брони с bookingEndTime < bookingStartTime
  - корректная установка статуса created
- Пример:
  - Input:
  - bookingStartTime = 2026-03-02T10:00
  - bookingEndTime = 2026-03-02T12:00
  - Expected:
  - bookingStatus = created
- Проверка пересечения бронирований
  - Система не должна позволять бронировать один и тот же слот.
  - Existing booking:
  - 10:00 — 12:00
  - New booking:
  - 11:00 — 13:00
  - Expected:
  - error = SLOT_ALREADY_RESERVED
  - Расчет доступных временных слотов
  - Working hours:
  - 09:00 — 18:00
  - Existing booking:
  - 12:00 — 13:00
  - Expected available slots:
  - 09:00–12:00
  - 13:00–18:00

2. Integration тестирование

Integration-тесты проверяют взаимодействие микросервисов.

Примеры сценариев:

Создание бронирования

Проверяется взаимодействие:

API Gateway

MS Booking

MS Availability

PostgreSQL

RabbitMQ

Сценарий:

Пользователь отправляет запрос на бронирование

Booking Service проверяет слот в Availability Service

Слот блокируется

Бронь сохраняется в PostgreSQL

Отправляется событие bookingCreated

Expected:

slot reserved

booking saved

event published

Оплата бронирования

Проверяется взаимодействие:

MS Payments

Payment Provider

RabbitMQ

MS Booking

Сценарий:

Пользователь инициирует оплату

Payments Service отправляет запрос провайдеру

Провайдер возвращает статус оплаты

Booking Service обновляет статус бронирования

Expected:

paymentStatus = completed

bookingStatus = paid

3. API тестирование

API тесты проверяют корректность REST-контрактов.

Примеры:

Регистрация пользователя

POST /v1/users

Expected:

HTTP 201

userId returned

Ошибка авторизации

POST /v1/auth/login

Expected:

HTTP 401

error = INVALID_CREDENTIALS

Получение списка бронирований

GET /v1/bookings?page=1&limit=10

Expected:

pagination returned

booking list returned

4. Нагрузочное тестирование

Load-тесты проверяют поведение системы при высокой нагрузке.

Примеры сценариев:

Поиск рабочих мест

1000 одновременных пользователей выполняют:

GET /v1/search

Проверяется:

среднее время ответа

нагрузка на PostgreSQL

эффективность Redis кэша

Expected:

response time < 2 seconds

no errors

Создание бронирований

Сценарий:

500 пользователей одновременно бронируют одно и то же место.

Проверяется:

защита от двойного бронирования

корректность блокировки слота

Expected:

1 booking created

499 requests rejected

Оплата бронирований

Сценарий:

200 одновременных платежей.

Проверяется:

устойчивость Payments Service

корректная обработка webhook

Expected:

payment status updated correctly

no lost events

Стратегия мониторинга бизнес-метрик:

Система мониторинга включает не только технические показатели (нагрузка, задержки, ошибки), но и бизнес-метрики, которые позволяют оценивать эффективность работы платформы бронирования. Сбор метрик осуществляется сервисом аналитики на основе событий системы (например: search, bookingCreated, paymentCompleted, checkInRegistered). Данные агрегируются и визуализируются в системах мониторинга и аналитики.

Конверсия бронирований

Метрика показывает долю пользователей, которые после выполнения поиска создают бронирование рабочего места.

Формула расчета:

Booking conversion rate = bookingCreated / search

Пример:

search events = 10000

bookingCreated = 1500

conversion = 15%

Метрика позволяет выявлять проблемы в процессе выбора и бронирования рабочих мест, например недостаточную доступность мест или неудобство пользовательского интерфейса.

Конверсия оплаты

Метрика показывает долю бронирований, которые были успешно оплачены пользователями.

Формула расчета:

Payment conversion = paymentCompleted / bookingCreated

Пример:

bookingCreated = 1500

paymentCompleted = 1200

conversion = 80%

Снижение этой метрики может указывать на проблемы в процессе оплаты, ошибки платежного провайдера или неудобный пользовательский сценарий оплаты.

Средний чек

Метрика показывает среднюю сумму платежа за одно бронирование.

Формула расчета:

Average order value = total payment amount / paymentCompleted

Пример:

total payment amount = 360000

paymentCompleted = 1200

average check = 300

Метрика используется для оценки доходности платформы и эффективности ценовой политики.

Загрузка рабочих мест

Метрика показывает долю времени, когда рабочие места заняты пользователями.

Формула расчета:

Occupancy rate = total booked time / total available time

Пример:

total booked time = 7200 часов

total available time = 10000 часов

occupancy rate = 72%

Метрика позволяет оценивать эффективность использования рабочих пространств и выявлять периоды низкой или высокой загрузки.

Количество бронирований

Метрика отражает общее количество созданных бронирований за определенный период времени.

Пример:

bookingCreated per day = 850

bookingCreated per month = 24000

Метрика используется для анализа роста системы и сезонных колебаний спроса.

Процент неявок (no-show)

Метрика показывает долю пользователей, которые забронировали место, но не пришли.

Формула расчета:

No-show rate = noShowDetected / bookingCreated

Пример:

bookingCreated = 1500

noShowDetected = 120

no-show rate = 8%

Метрика помогает выявлять проблемы с дисциплиной пользователей и может использоваться для внедрения штрафов или предоплаты.

Все бизнес-метрики формируются на основе событий системы, сохраняются в аналитическом хранилище и отображаются на дашбордах мониторинга для анализа состояния платформы и принятия продуктовых решений.

User goals:

удаленно выбирать конфиденциальное рабочее место

удаленно выбирать рабочее место по бюджету

удаленно выбирать ближайшее рабочее место

удаленно быстро бронировать рабочее место

удаленно оплачивать забронированное рабочее место

иметь доступ к деталям бронирования

иметь доступ к описанию доступных рабочих мест

User stories

1. Как пользователь, я хочу находить подходящее рабочее место по различным параметрам, чтобы быстро выбрать удобное место для работы.

   Описание:

   Пользователь хочет найти рабочее место, соответствующее его требованиям. Он должен иметь возможность искать пространства по городу и карте, фильтровать их по доступности, расстоянию и характеристикам, а также просматривать подробную информацию перед выбором.

   Критерии приемки:

   Пользователь может искать рабочие места по городу.

   Пользователь может видеть рабочие пространства на карте.

   Пользователь может фильтровать результаты по доступности и параметрам.

   Пользователь может просматривать подробную информацию о рабочем месте.

   Пользователю отображаются только доступные варианты.

   Результаты поиска отображаются за приемлемое время.

   Дополнительные требования:

   Поиск должен выполняться быстро.

   Данные о доступности должны быть актуальными.

   Интерфейс должен быть понятным и удобным.

2. Как пользователь, я хочу забронировать рабочее место онлайн, чтобы гарантировать его доступность в нужное время.

   Описание:

После выбора рабочего места пользователь должен иметь возможность забронировать его на определённое время. Система должна сохранить бронь и предоставить пользователю доступ к её деталям.

Критерии приемки:

Пользователь может выбрать дату и время бронирования.

Пользователь может подтвердить бронь.

Система сохраняет бронь и обновляет доступность места.

Пользователь может просмотреть детали бронирования.

Пользователь может отменить бронь при необходимости.

Дополнительные требования

Система не должна допускать двойного бронирования.

Статус бронирования должен обновляться в реальном времени.

Бронь должна сохраняться даже при кратковременных сбоях соединения.

3. Как пользователь я хочу оплатить забронированное место используя онлайн или оффлайн методы

Описание:

Пользователь должен иметь возможность оплатить бронирование онлайн или выбрать оплату на месте. Система должна корректно обрабатывать статусы оплаты и информировать пользователя о результате.

Критерии приемки:

Пользователь может выбрать способ оплаты.

Пользователь может оплатить бронь банковской картой или через СБП.

Пользователь может выбрать оплату на месте.

Система обновляет статус оплаты после транзакции.

Пользователь получает уведомление о результате оплаты.

Пользователь может повторить попытку оплаты при неудаче.

Дополнительные требования:

Платёжные операции должны выполняться безопасно.

Статус оплаты должен сохраняться корректно.

Система должна корректно обрабатывать ошибки платёжного провайдера.

Use case: Выбор рабочего места

Актор: пользователь

Цель: удалённо выбрать подходящее рабочее место

Триггер: Пользователь решает найти рабочее место.

Предусловие:

Пользователь авторизован.

Система доступна.

Данные о рабочих местах актуальны.

Основной поток:

1. Пользователь открывает сайт.
2. Пользователь вводит название города и дату планируемой брони.
3. Система выполняет поиск по заданным параметрам.
4. Система формирует список подходящих рабочих мест.
5. Система отображает результаты поиска.
6. Пользователь просматривает найденные варианты.

Альтернативные потоки:

1. Ничего не найдено

1.1 Система сообщает, что подходящие варианты не найдены.

1.2 Система предлагает изменить критерии.

1.3 Пользователь изменяет параметры поиска.

2. Ошибка соединения или сервера

2.1 Система отображает сообщение об ошибке.

2.2 Пользователь может повторить попытку.

3: Пользователь отменяет поиск

3.1 Пользователь покидает экран поиска.

3.2 Сценарий завершается.

Бизнес-правила

Отображаются только доступные рабочие места.

Данные о доступности должны быть актуальными.

Результаты сортируются по релевантности.

Результат

Пользователь получает список рабочих мест, соответствующих заданным критериям, и может выбрать подходящий вариант.

Use case: Бронь рабочего места

Актор: пользователь

Цель: забронировать выбранное рабочее место

Триггер: Пользователь решает забронировать рабочее место.

Предусловие:

Пользователь авторизован.

Система доступна.

Данные о рабочих местах актуальны.

Пользователь выбрал рабочее место

Основной поток:

1. Пользователь открывает страницу с выбранным рабочим местом
2. Пользователь нажимает кнопку "Забронировать" на странице рабочего места
3. Система перенаправляет пользователя на страницу с деталями бронирования
4. Пользователь проверяет данные и подтверждает бронирование
5. Система сохраняет бронирование и обновляет информацию о доступности рабочего места в базе данных
6. Система перенаправляет пользователя на страницу с подтвержденным бронированием и его деталями

Альтернативные потоки:

1. Данные не верны
    1.1 Пользователь изменяет данные бронирования
    1.2 Система проверяет корректность обновленных данных
    1.3 Система обновляет информацию на странице
    1.4 Пользователь подтверждает бронирование
2. Пользователь передумал бронировать
    2.1 Пользователь закрывает страницу с деталями бронирования
    2.2 Сценарий завершается.
3. Ошибка соединения или сервера
    3.1 Система отображает сообщение об ошибке.
    3.2 Пользователь может повторить попытку.

Результат:

Пользователь забронировал подходящий вариант может просмотреть детали бронирования и перейти к оплате

USE Case: Оплата забронированного места

Актор: Пользователь

Цель: Оплатить бронь рабочего места

Триггер: Пользователь решает оплатить бронь рабочего места

Предусловие:

Пользователь авторизован.

Система доступна.

Данные о рабочих местах актуальны.

Пользователь забронировал рабочее место.

Основной поток:

1. Пользователь нажимает кнопку оплаты
2. Система перенаправляет пользователя на страницу выбора способа оплаты
3. Пользователь выбирает оплату картой
4. Пользователь вводит данные карты
5. Пользователь нажимает кнопку далее
6. Система перенаправляет пользователя на страницу банка для подтверждения оплаты
7. Пользователь подтверждает оплату
8. Система обновляет данные о статусе оплаты в базе данных
9. Система перенаправляет пользователя на страницу с оплаченным бронированием и его деталями

Альтернативные потоки:

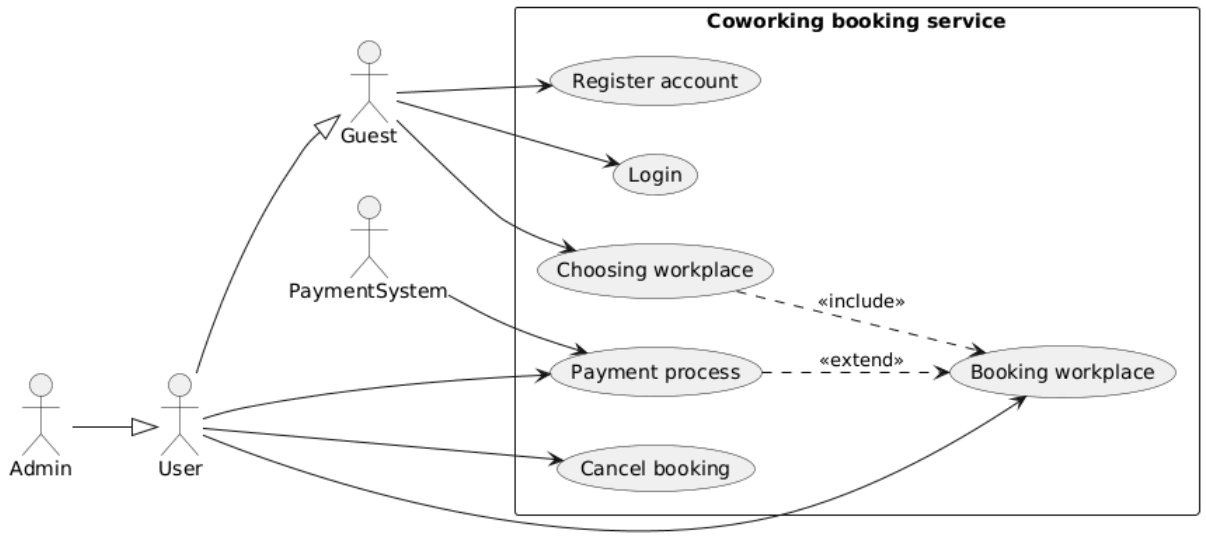
1. Пользователь выбирает "оплату на месте":

   1.1 Система оставляет статус NOT_PAID и указывает выбор оффлайн-оплаты
2. Платеж отклонен провайдером(неверные данные/недостаточно средств и т.п.)

   2.1 Система показывает сообщение об ошибке

   2.2 Система предлагает повторить попытку

UML Use Case diagram




```
@startuml
left to right direction

Actor Admin
Actor User
Actor Guest
Actor PaymentSystem
rectangle "Coworking booking service"{
'Use Cases'
usecase "Cancel booking" as UC_CancelB
usecase "Choosing workplace" as UC_Choose
usecase "Booking workplace" as UC_Booking
usecase "Payment process" as UC_Payment
usecase "Register account" as UC_Register
usecase "Login" as UC_Login
}

User --|> Guest
Admin --|> User

'Actors with use case assotioations'
Guest --> UC_Register
Guest --> UC_Login
Guest --> UC_Choose
```

```
User --> UC_Booking
User --> UC_CancelB
User --> UC_Payment
PaymentSystem --> UC_Payment
UC_Choose ..>UC_Booking: <<include>>
UC_Payment ..>UC_Booking: <<extend>>
@enduml
```

UML Booking state diagram:

@startuml

[*] --> Created

Created --> Reserved : user confirms booking

Reserved --> AwaitingOnlinePayment : user selects online payment

Reserved --> PayOnSite : user selects offline payment

AwaitingOnlinePayment --> Paid : payment provider confirms payment

AwaitingOnlinePayment --> Unpaid : payment provider returns failure

AwaitingOnlinePayment --> Cancelled : after 15 min / system cancels unpaid booking

Unpaid --> Paid : user retries payment and payment confirmed

Unpaid --> Cancelled : user cancels booking

Unpaid --> Cancelled : after 15 min / system cancels unpaid booking

PayOnSite --> InUse : user arrives and pays

PayOnSite --> NotUsed : after bookingStartTime + 15 min / system marks no-show

Paid --> InUse : user arrives

Paid --> NotUsed : after bookingStartTime + 15 min / system marks no-show

Paid --> Cancelled : user cancels booking before start time

InUse --> Completed : bookingEndTime reached / system closes booking

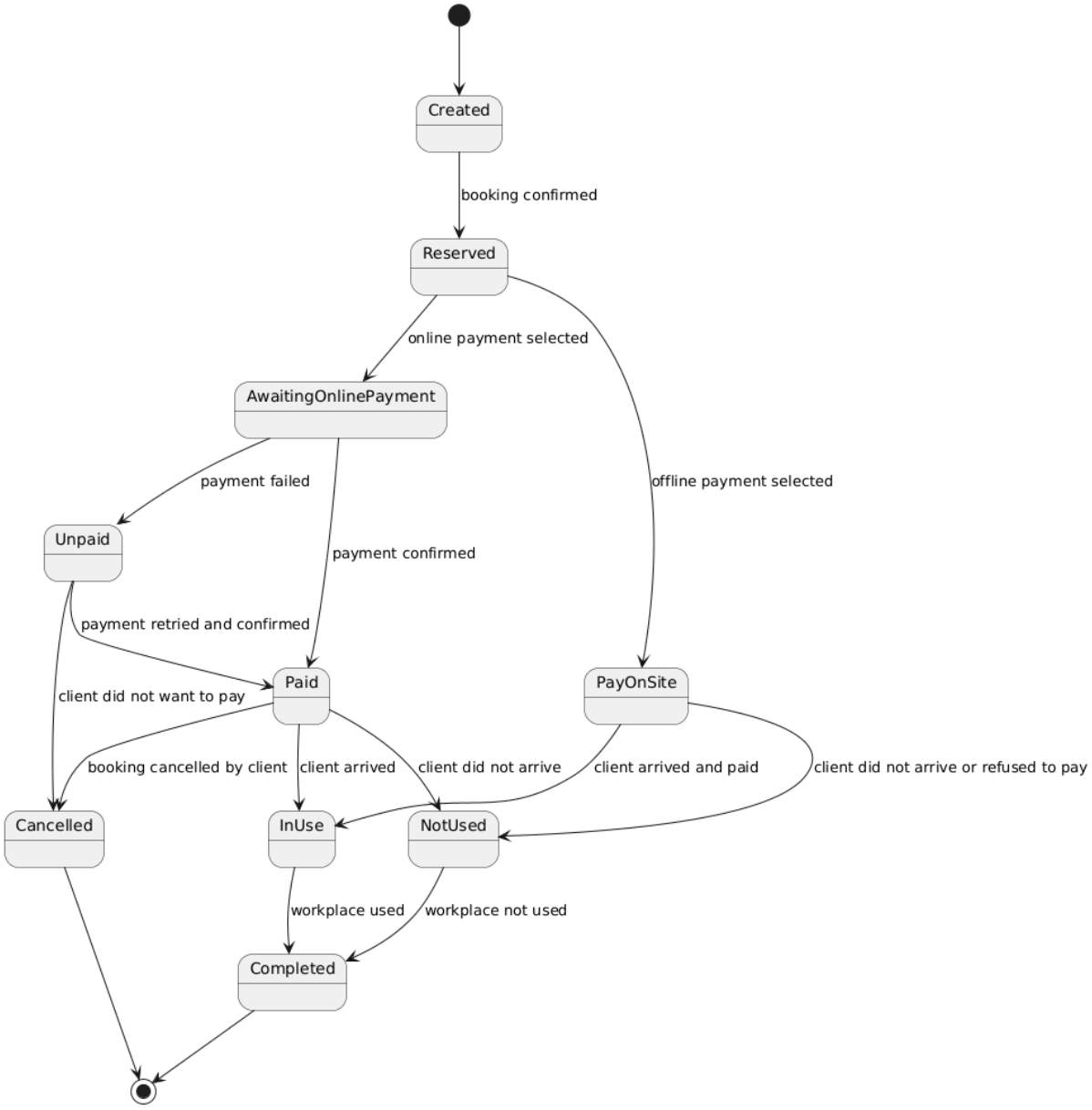
NotUsed --> Completed : system finalizes booking

Cancelled --> [*]

Completed --> [*]

@enduml

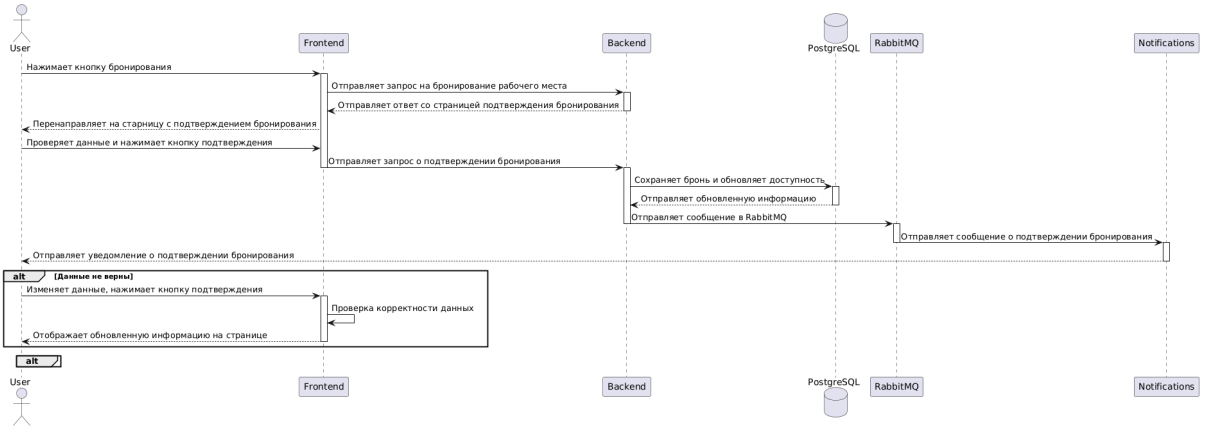
UML Class diagram




```
@startuml
class Place {
  placeId
  placeLocation
  placeCapacity
  placeName
  placeType
  placeDescription
}
class User {
```

```
  userId
  firstName
  lastName
  email
  phone
  password
  auth
}
class Booking {
  bookingId
  bookingStatus
  bookingStartTime
  bookingEndTime
}
class Payment {
  paymentId
  paymentStatus
  paymentMethod
  paymentProvider
  paymentDate
}
User "1" --> "0..*" Booking : creates
Booking "1" --> "1" Place : reserves
Booking "1" *-- "0..1" Payment
@enduml
```

UML Sequence diagram "Пользователь бронирует рабочее место"



@startuml

actor User

participant Frontend

participant Backend

database PostgreSQL

participant RabbitMQ

participant Notifications

User -> Frontend: Нажимает кнопку бронирования

activate Frontend

Frontend -> Backend: Отправляет запрос на бронирование рабочего места

activate Backend

Backend --> Frontend: Отправляет ответ со страницей подтверждения бронирования

deactivate Backend

Frontend --> User: Перенаправляет на старницу с подтверждением бронирования

User -> Frontend: Проверяет данные и нажимает кнопку подтверждения

Frontend -> Backend: Отправляет запрос о подтверждении бронирования

deactivate Frontend

activate Backend

Backend -> PostgreSQL: Сохраняет бронь и обновляет доступность

activate PostgreSQL

PostgreSQL --> Backend: Отправляет обновленную информацию

deactivate PostgreSQL

Backend -> RabbitMQ: Отправляет сообщение в RabbitMQ

deactivate Backend

activate RabbitMQ

RabbitMQ -> Notifications: Отправляет сообщение о подтверждении бронирования
deactivate RabbitMQ
activate Notifications
Notifications --> User: Отправляет уведомление о подтверждении бронирования
deactivate Notifications

alt Данные не верны
User -> Frontend: Изменяет данные, нажимает кнопку подтверждения
activate Frontend
Frontend -> Frontend: Проверка корректности данных
Frontend --> User: Отображает обновленную информацию на странице
deactivate Frontend
end
alt
end
@enduml

Redis

Используется на проекте для повышения производительности и масштабируемости система использует Redis как in-memory хранилище данных.

Redis применяется для ускорения обработки запросов, снижения нагрузки на PostgreSQL и обеспечения стабильной работы при высокой нагрузке.

В Redis кэшируются следующие данные:

Результаты поиска рабочих мест

ключ: search:{city}:{date}:{page}:{filtersHash}

TTL: 30–60 секунд

Информация о рабочем месте

ключ: place:{placeId}

TTL: 5–10 минут

Отзывы и рейтинг

ключи:

place:{placeId}:reviews

place:{placeId}:rating

TTL: 2–5 минут

В Redis не кэшируются критически важные и часто изменяемые данные:

бронирования;

платежные операции и статусы оплат;

check-in / check-out;

текущая занятость рабочих мест в реальном времени;

webhook-события.

Источник истины для этих данных — PostgreSQL.

Система использует стратегию Cache-Aside:

API проверяет наличие данных в Redis.

При наличии — данные возвращаются из кэша.

При отсутствии — выполняется запрос к PostgreSQL.

Полученные данные сохраняются в Redis.

Ответ возвращается клиенту.

Инвалидация кэша

Кэш автоматически сбрасывается при изменении данных:

обновление информации о рабочем месте → удаление place:{placeId};

добавление или изменение отзывов → удаление place:{placeId}:reviews и place: {placeId}:rating;

изменение доступности рабочих мест → очистка связанных ключей поиска;

изменение рейтинга → обновление агрегированных значений.

Redis используется для защиты API:

ограничение частоты запросов (rate limiting);

защита от brute-force атак при входе;

контроль попыток авторизации.

Примеры ключей:

rate:login:{ip}

rate:api:{userId}

Консистентность данных

Redis является временным слоем хранения данных.

основной источник данных — PostgreSQL;

модель согласованности: eventual consistency;

кэш используется только для ускорения чтения.

Redis Pub/Sub используется для передачи событий:

обновление доступности рабочих мест;

изменения статуса бронирования;

уведомления в реальном времени.

Архитектура

Система построена на микросервисной архитектуре. Каждый сервис отвечает за отдельную бизнес-область и взаимодействует с другими сервисами через REST API и асинхронные события.

MS Auth

- Отвечает за аутентификацию и управление доступом.
- Функции:
  - выдача JWT access и refresh токенов
  - интеграция с внешними OIDC провайдерами
  - хранение хэшей паролей
  - управление сессиями и refresh-токенами
- JWT Token Lifecycle
  - Система использует два типа токенов:
    - Access Token
    - Refresh Token
  - Access Token:
    - TTL: 15 минут
  - Refresh Token:
    - TTL: 7 дней
  - Refresh rotation:
    - При каждом обновлении access token
    - выдается новый refresh token.
    - Предыдущий refresh token становится недействительным.
  - Logout:
    - При выходе refresh token помещается в blacklist в Redis.
    - Blacklist TTL соответствует сроку жизни токена.

MS User

- Управляет данными пользователя.
- Функции:
  - управление профилем
  - хранение контактных данных
  - настройки пользователя

MS Place

- Управляет рабочими пространствами.

Функции:

- хранение информации о местах
- изменение данных мест
- управление характеристиками и описаниями
- удаление и создание мест

MS Availability

Отвечает за доступность рабочих мест и временных слотов.

Функции:

- расчет доступных временных слотов
- блокировка слота при бронировании
- освобождение слота при отмене
- обработка текущей занятости
- применение правил доступности

MS Booking

Управляет жизненным циклом бронирования.

Функции:

- создание бронирования
- отмена брони
- предотвращение двойного бронирования
- управление статусами бронирования

MS Payments

Управляет платежными операциями.

Функции:

- инициализация платежа
- интеграция с банками и платежными провайдерами
- обработка статусов оплаты
- возвраты средств
- хранение платежной информации

Система не хранит данные банковских карт.

Для обработки платежей используется

редирект на страницу платежного провайдера.

Обработка карт выполняется платежным провайдером

Система получает только:

- paymentId
- providerTransactionId
- paymentStatus

Все операции соответствуют требованиям PCI DSS.

MS Notifications

Управляет уведомлениями пользователей.

Функции:

- отправка уведомлений
- отмена отправки
- повторная отправка (retry)
- поддержка каналов доставки (email, push, SMS)

MS Search

Обеспечивает быстрый поиск рабочих мест.

Функции:

- поиск по параметрам
- фильтрация
- сортировка
- использование кэша для ускорения результатов

MS Analytics

Отвечает за сбор и анализ данных системы.

Функции:

- сбор событий
- формирование отчетов
- аналитика использования
- экспорт и выгрузка данных

MS API Gateway

Единая точка входа для клиентских приложений. Обрабатывает маршрутизацию запросов, авторизацию и агрегацию ответов.

Функции

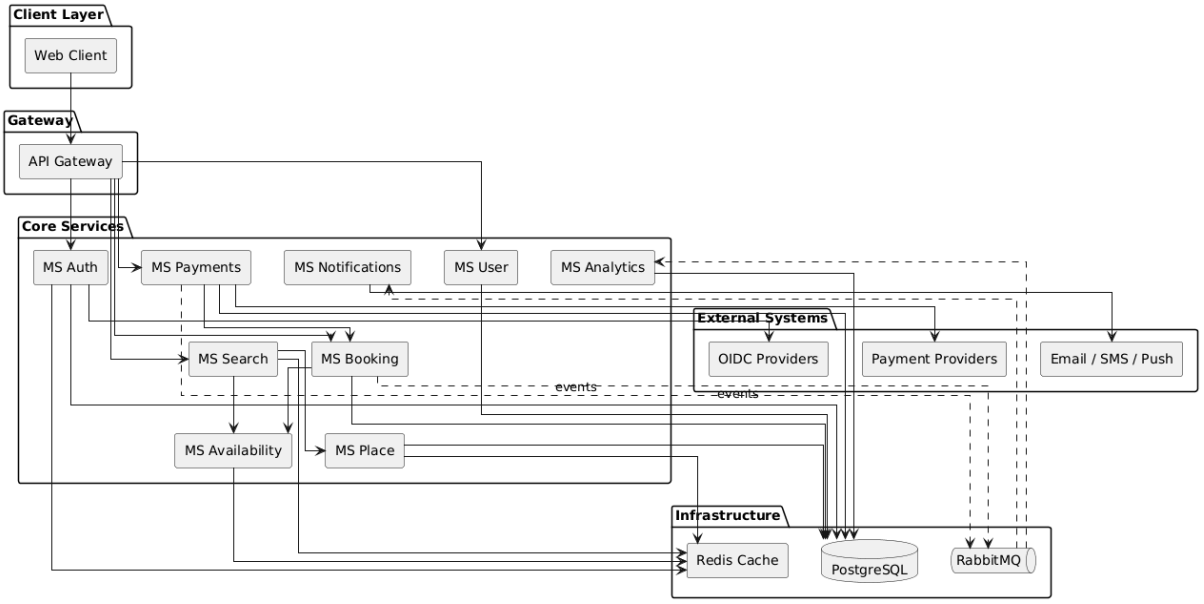
- маршрутизация запросов к микросервисам
- проверка JWT и передача пользовательского контекста
- rate limiting и базовая защита API
- агрегация данных из нескольких сервисов

кэширование публичных ответов (опционально)

Взаимодействие микросервисов

Сервисы взаимодействуют синхронно через REST API и асинхронно через брокер сообщений (RabbitMQ).
Асинхронные события используются для уведомлений, аналитики и обновлений в реальном времени.

UML Component Diagram




```
@startuml
skinparam componentStyle rectangle
skinparam shadowing false
skinparam linetype ortho
package "Client Layer" {
  [Web Client]
}
package "Gateway" {
  [API Gateway]
}
package "Core Services" {
  [MS Auth]
  [MS User]
  [MS Search]
  [MS Booking]
  [MS Availability]
```

```
  [MS Place]
  [MS Payments]
  [MS Notifications]
  [MS Analytics]
}
package "Infrastructure" {
  database "PostgreSQL" as DB
  component "Redis Cache" as Redis
  queue "RabbitMQ" as MQ
}
package "External Systems" {
  [OIDC Providers]
  [Payment Providers]
  [Email / SMS / Push]
}
[Web Client] --> [API Gateway]
[API Gateway] --> [MS Auth]
[API Gateway] --> [MS User]
[API Gateway] --> [MS Search]
[API Gateway] --> [MS Booking]
[API Gateway] --> [MS Payments]
[MS Search] --> [MS Place]
[MS Search] --> [MS Availability]
[MS Booking] --> [MS Availability]
[MS Payments] --> [MS Booking]
[MS Booking] ..> MQ : events
[MS Payments] ..> MQ : events
MQ ..> [MS Notifications]
MQ ..> [MS Analytics]
[MS Auth] --> DB
[MS User] --> DB
[MS Place] --> DB
[MS Booking] --> DB
[MS Payments] --> DB
[MS Analytics] --> DB
```

[MS Search] --> Redis

[MS Place] --> Redis

[MS Availability] --> Redis

[MS Auth] --> Redis

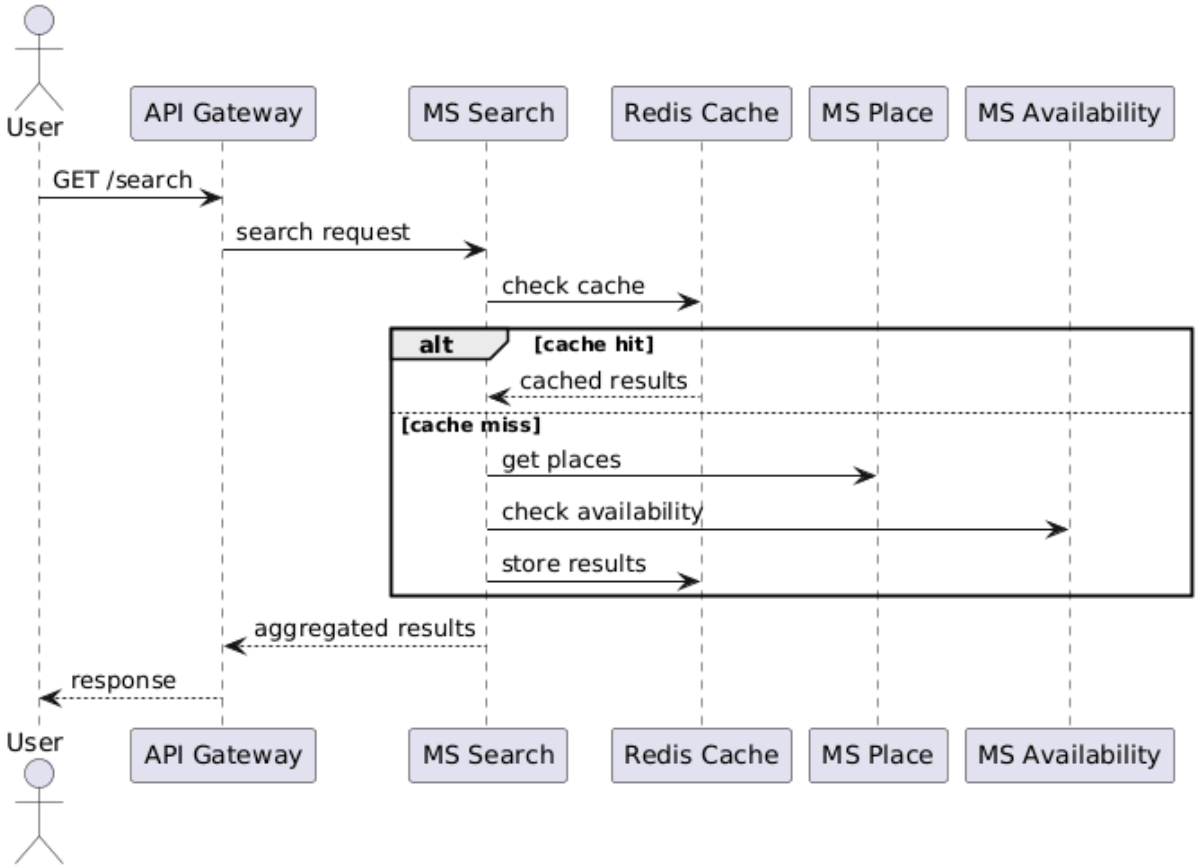
[MS Auth] --> [OIDC Providers]

[MS Payments] --> [Payment Providers]

[MS Notifications] --> [Email / SMS / Push]

@enduml

UML Sequence Diagram: Поиск рабочего места



@startuml

actor User

participant "API Gateway" as Gateway

participant "MS Search" as Search

participant "Redis Cache" as Redis

participant "MS Place" as Place

participant "MS Availability" as Availability

User -> Gateway: GET /search

Gateway -> Search: search request

Search -> Redis: check cache

alt cache hit

  Redis --> Search: cached results

else cache miss

  Search -> Place: get places

  Search -> Availability: check availability

  Search -> Redis: store results

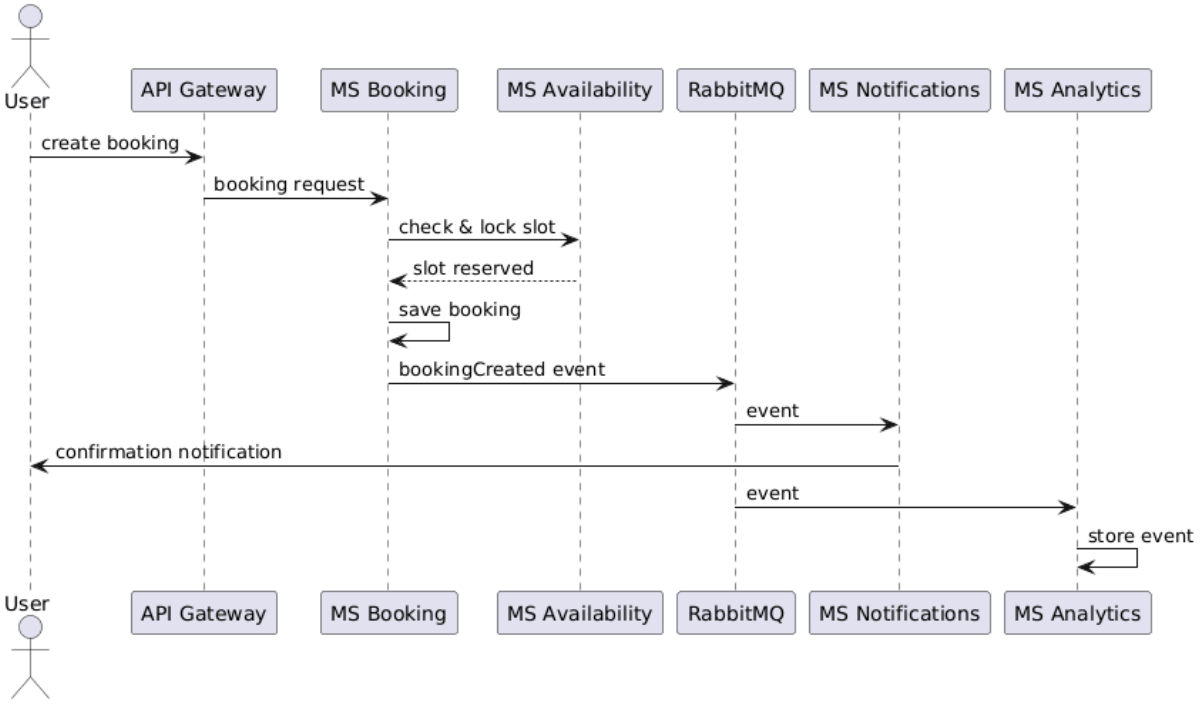
end

Search --> Gateway: aggregated results

Gateway --> User: response

@enduml

UML Sequence Diagram: Бронирование рабочего места



@startuml

actor User

participant "API Gateway" as Gateway

participant "MS Booking" as Booking

participant "MS Availability" as Availability

participant "RabbitMQ" as MQ

participant "MS Notifications" as Notifications

participant "MS Analytics" as Analytics

User -> Gateway: create booking

Gateway -> Booking: booking request

Booking -> Availability: check & lock slot

Availability --> Booking: slot reserved

Booking -> Booking: save booking

Booking -> MQ: bookingCreated event

MQ -> Notifications: event

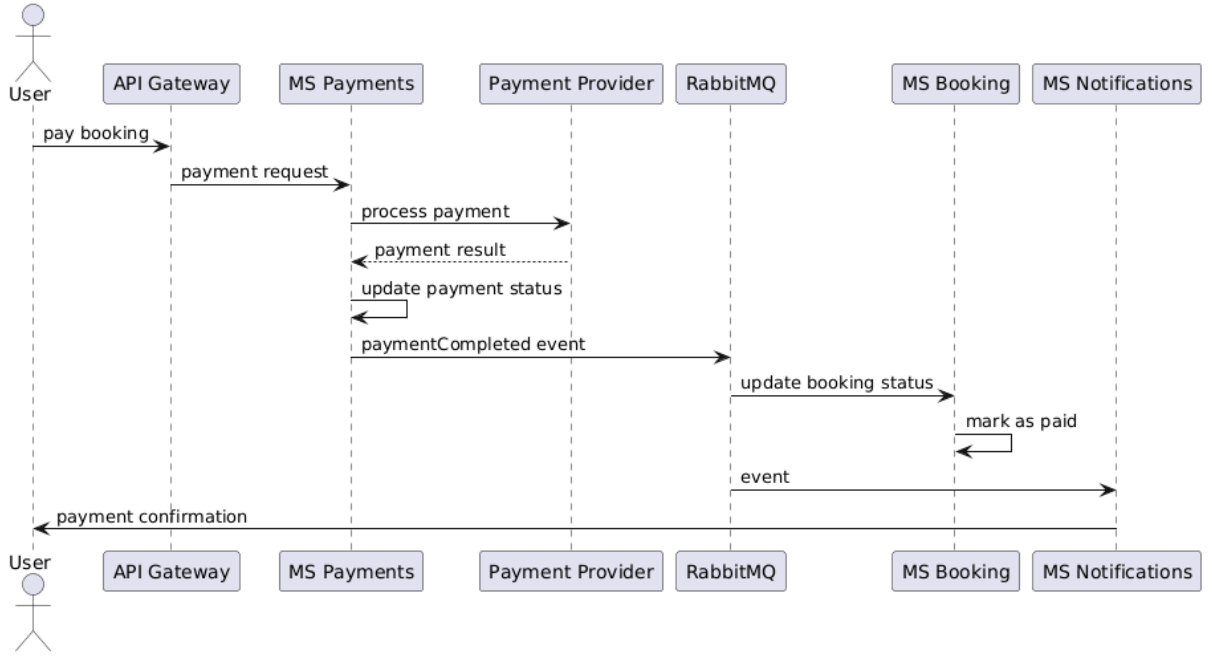
Notifications -> User: confirmation notification

MQ -> Analytics: event

Analytics -> Analytics: store event

@enduml

UML Sequence Diagram: Оплата бронирования



@startuml

```
actor  User
participant "API Gateway" as Gateway
participant "MS Payments" as Payments
participant "Payment Provider" as Provider
participant "RabbitMQ" as MQ
participant "MS Booking" as Booking
participant "MS Notifications" as Notifications
User -> Gateway: pay booking
Gateway -> Payments: payment request
Payments -> Provider: process payment
Provider --> Payments: payment result
Payments -> Payments: update payment status
Payments -> MQ: paymentCompleted event
MQ -> Booking: update booking status
Booking -> Booking: mark as paid
MQ -> Notifications: event
Notifications -> User: payment confirmation
@enduml
```

Брокер сообщений RabbitMQ

Используется для уведомлений пользователей (push, email, SMS, мессенджеры).

Очереди:

NotificationsQueue

EventsQueue

Пример сообщения:

```
{
"userId": 123,
"type": "bookingConfirmed",
"bookingId": 456,
"timestamp": "2026-01-01T12:30:00",
"message": "Ваша бронь подтверждена"
}
```

Типы уведомлений:

Бронирование:

bookingConfirmed — бронь подтверждена

bookingCancelled — бронь отменена

bookingReminder — напоминание о брони

bookingModified — бронь изменена

bookingExpired — бронь отменена из-за неоплаты

Оплата:

paymentSuccessful — оплата выполнена

paymentFailed — ошибка оплаты

refundProcessed — возврат средств выполнен

Посещение:

checkInSuccessful — пользователь отметился при входе

noShowRecorded — зафиксирована неявка

Системные уведомления:

newWorkspaceNearby — новое пространство поблизости

promotionAvailable — доступна акция или скидка

EventsQueue:

Используется для передачи доменных событий между сервисами.

Эти события не предназначены напрямую для пользователя.

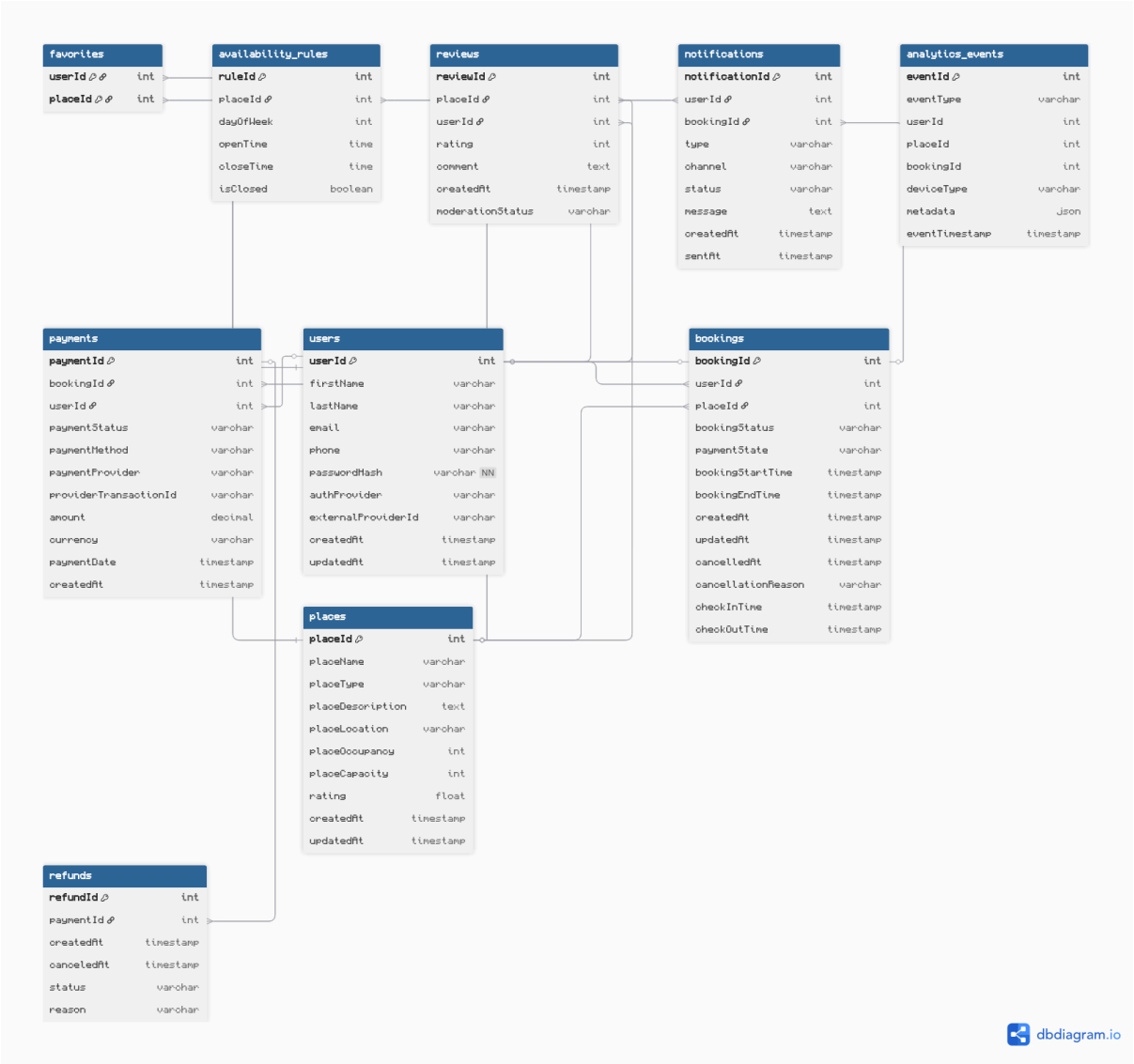
Пример сообщения:

{
"type": "bookingConfirmed",
"bookingId": 456,
"userId": 123,
"placeId": 88,
"startTime": "2026-01-02T09:00:00",
"endTime": "2026-01-02T18:00:00",
"timestamp": "2026-01-01T12:30:00"
}

Типы событий:

- События бронирования:
  - bookingCreated
  - bookingConfirmed
  - bookingCancelled
  - bookingModified
  - bookingExpired
  - bookingCompleted
  - noShowDetected
- События оплаты:
  - paymentInitiated
  - paymentCompleted
  - paymentFailed
  - refundRequested
  - refundCompleted
- События использования:
  - checkInRegistered
  - checkOutRegistered
  - workspaceOccupied
  - workspaceReleased
- События доступности:
  - availabilityUpdated
  - capacityThresholdReached
- Системные события:
  - notificationRequested
  - analyticsEventRecorded

База данных PostgreSQL:



users

- userId PRIMARY KEY
- firstName
- lastName
- email UNIQUE
- phone UNIQUE
- passwordHash CHECK (char_length(passwordHash) > 0)
- authProvider CHECK (authProvider IN ('local','google','vk'))
- externalProviderId
- createdAt
- updatedAt

INDEX idx_users_email (email)

INDEX idx_users_phone (phone)

INDEX idx_users_externalProviderId (externalProviderId)

places

placeId PRIMARY KEY

placeName

placeType

placeDescription

placeLocation

placeOccupancy CHECK (placeOccupancy >= 0)

placeCapacity CHECK (placeCapacity > 0)

rating CHECK (rating BETWEEN 0 AND 5)

createdAt

updatedAt

INDEX idx_places_type (placeType)

INDEX idx_places_rating (rating)

INDEX idx_places_location (placeLocation)

bookings

bookingId PRIMARY KEY

userId FOREIGN KEY → users.userId

placeId FOREIGN KEY → places.placeId

bookingStatus CHECK (bookingStatus IN

('created','reserved','paid','cancelled','completed','no_show'))

paymentState CHECK (paymentState IN ('pending','paid','failed','refunded'))

bookingStartTime

bookingEndTime CHECK (bookingEndTime > bookingStartTime)

createdAt

updatedAt

cancelledAt

cancellationReason

checkInTime

checkOutTime

INDEX idx_bookings_user (userId)

INDEX idx_bookings_place (placeId)

INDEX idx_bookings_status (bookingStatus)

INDEX idx_bookings_user_status (userId, bookingStatus)

INDEX idx_bookings_place_time (placeId, bookingStartTime, bookingEndTime)

payments

paymentId PRIMARY KEY

bookingId FOREIGN KEY → bookings.bookingId

userId FOREIGN KEY → users.userId

paymentStatus CHECK (paymentStatus IN ('pending','completed','failed'))

paymentMethod CHECK (paymentMethod IN ('card','sbp','on_site'))

paymentProvider CHECK (paymentProvider IN ('stripe','bank','sbp'))

providerTransactionId

amount CHECK (amount > 0)

currency

paymentDate

createdAt

INDEX idx_payments_booking (bookingId)

INDEX idx_payments_user (userId)

INDEX idx_payments_providerTransactionId (providerTransactionId)

notifications:

notificationId PRIMARY KEY

userId FOREIGN KEY → users.userId

bookingId FOREIGN KEY → bookings.bookingId

type CHECK (type IN ('bookingConfirmed','reminder','paymentFailed'))

channel CHECK (channel IN ('push','email','sms'))

status CHECK (status IN ('pending','sent','failed'))

message

createdAt

sentAt

INDEX idx_notifications_user (userId)

INDEX idx_notifications_booking (bookingId)

INDEX idx_notifications_status (status)

reviews:

reviewId PRIMARY KEY

placeId FOREIGN KEY → places.placeId

userId FOREIGN KEY → users.userId

rating CHECK (rating BETWEEN 1 AND 5)

comment

createdAt

moderationStatus CHECK (moderationStatus IN ('pending','approved','rejected'))

INDEX idx_reviews_place (placeId)

INDEX idx_reviews_user (userId)

favorites:

userId FOREIGN KEY → users.userId

placeId FOREIGN KEY → places.placeId

PRIMARY KEY (userId, placeId)

INDEX idx_favorites_user (userId)

INDEX idx_favorites_place (placeId)

analytics_events:

eventId PRIMARY KEY

eventType CHECK (eventType IN ('search','bookingCreated','paymentCompleted'))

userId

placeId

bookingId

deviceType

metadata JSON

eventTimestamp

INDEX idx_analytics_eventType (eventType)

INDEX idx_analytics_user (userId)

INDEX idx_analytics_booking (bookingId)

INDEX idx_analytics_time (eventTimestamp)

availability_rules:

ruleId PRIMARY KEY

placeId FOREIGN KEY → places.placeId

dayOfWeek CHECK (dayOfWeek BETWEEN 1 AND 7)

openTime

closeTime CHECK (openTime < closeTime)

isClosed

INDEX idx_availability_place (placeId)

refunds:

refundId PRIMARY KEY

paymentId FOREIGN KEY → payments.paymentId

createdAt

canceledAt

status CHECK (status IN ('pending','completed','failed'))

reason

INDEX idx_refunds_payment (paymentId)

INDEX idx_refunds_status (status)

audit_logs:

auditId PRIMARY KEY

eventType CHECK (eventType IN

('account_deleted','password_changed','email_changed','refund_requested','refund_c

ompleted','admin_login','failed_login'))

actorUserId FOREIGN KEY → users.userId

targetUserId FOREIGN KEY → users.userId

ipAddress

userAgent

metadata JSON

createdAt NOT NULL

INDEX idx_audit_logs_eventType (eventType)

INDEX idx_audit_logs_actorUserId (actorUserId)

INDEX idx_audit_logs_targetUserId (targetUserId)

INDEX idx_audit_logs_createdAt (createdAt)

INDEX idx_audit_logs_actor_time (actorUserId, createdAt)

REST API

HOST: https://cobooking.com/api

/v1/users

1.Регистрация:

Метод: POST

URL: /v1/users

Заголовки:

Content-Type: application/json

Запрос:

```
{
"firstName": "Ivan",
"lastName": "Ivanov",
"email": "user@gmail.com",
"password": "randompassword",
"address": "Country, City, Street, House"
}
```

Ответ:

```
HTTP/1.1 201 Created
Content-Type: application/json
{
"userId": 11,
"firstName": "Ivan",
"lastName": "Ivanov",
"email": "user@gmail.com"
}
```

Ошибки:

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
{
"error": "BAD_REQUEST"
"message": "Введен некорректный email"
}

HTTP/1.1 409 Conflict
```

Content-Type: application/json

{

"error": "CONFLICT"

"message": "Email уже зарегистрирован"

}

2.Вход:

Метод: POST

URL: /v1/auth/login

Заголовки:

Content-Type: application/json

Запрос:

{

"email": "user@gmail.com",

"password": "randompassword"

}

Ответ:

HTTP/1.1 200 OK

{

"accessToken": <randomJWTtoken>,

"refreshToken":<<randomRefreshToken>>

}

Ошибки:

HTTP/1.1 401 Unauthorized

Content-Type: application/json

{

"error": "INVALID_CREDENTIALS",

"message": "Неверный email или пароль"

}

HTTP/1.1 429 Too Many Requests

Content-Type: application/json

{

"error": "ACCOUNT_BLOCKED",

"message": "Слишком много неудачных попыток. Попробуйте позже."

}

3.Удаление аккаунта:

Метод: DELETE

URL: /v1/users/{userId}

Заголовки:

Authorization: Bearer <randomJWTtoken>

Ответ:

HTTP/1.1 200 OK

Content-Type: application/json

{

"message": "Account successfully deleted"

}

Ошибки:

HTTP/1.1 404 Not Found

Content-Type: application/json

{

"error": "NOT_FOUND"

"message": "User does not exist"

}

4. Проверка авторизации

Метод: GET

URL: /v1/auth/check

Заголовки:

Authorization: Bearer <randomJWTtoken>

Ответ:

HTTP/1.1 200 OK

Content-Type: application/json

{

"authenticated": true,

"userId": 11,

"role": "user",

"tokenExp": "2026-03-06T14:00:00Z"

}

1.5 Авторизация через Google

Запрос:

Метод: POST

URL: /v1/auth/google

Заголовки:

Content-Type: application/json

{

"code": <randomcode>

}

Ответ:

HTTP/1.1 200 OK

Content-Type: application/json

{

"userId": 123

"email": "user@gmail.com"

"token": <randomtoken>

}

Ошибки:

HTTP/1.1 400 Bad Request

Content-Type: application/json

{

"error": "OAUTH_CODE_INVALID",

"message": "Неверный код авторизации"

}

HTTP/1.1 401 Unauthorized

Content-Type: application/json

{

"error": "OAUTH_AUTH_FAILED",

"message": "Ошибка авторизации через провайдера"

}

1.6 Авторизация через VK

Запрос:

Метод: POST

URL: /v1/auth/vk

Заголовки:

Content-Type: application/json

{

"code": "vk_authorization_code"

}

Ответ:

HTTP/1.1 200 OK

Content-Type: application/json

{

"userId": 18,

"email": "user@vk.com",

"token": <randomtoken>

}

Ошибки:

HTTP/1.1 400 Bad Request

Content-Type: application/json

{

"error": "OAUTH_CODE_INVALID",

"message": "Неверный код авторизации"

}

HTTP/1.1 401 Unauthorized

Content-Type: application/json

{

"error": "OAUTH_AUTH_FAILED",

"message": "Ошибка авторизации через провайдера"

}

6. Получение информации о пользователе

Метод: GET

URL: /v1/users/{userId}

Заголовки:

Authorization: Bearer <randomJWTtoken>

Ответ:

HTTP/1.1

Content-Type: application/json

{

"userId": 11

"firstName": "Ivan",

"lastName": "Ivanov",

"email": "user@gmail.com"

}

Ошибки:

HTTP/1.1 404 Not Found

Content-Type: application/json

{

"error": "NOT_FOUND"

"message": "User does not exist"

}

/v1/payments

1.Создание Платежа

Запрос:

Метод: POST

URL: /v1/payments/pay

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTtoken>

{

"bookingId": 22,

"paymentMethod": "card",

"amount": 1299.99,

"currency": "RUB"

}

Ответ:

HTTP/1.1 201 Created

Content-Type: application/json

{

"paymentId": 234,

"bookingId": 22,

"userId": 11,

"paymentStatus": "pending",

"paymentMethod": "card",

"createdAt": 2026-02-28T14:30:00Z,

"amount": 1299.99,

"currency": RUB,

"paymentProvider": "Sberbank",

"providerTransactionId":e3f9a2d1-7c4a-4b90-9a1c-8d3c6f5b
2a11,
}

Ошибки:

HTTP/1.1 400 Bad Request
Content-Type: application/json
{
"error": "BOOKING_ALREADY_PAID",
"message": "This booking has already been paid"
}
{
"error": "BOOKING_CANCELLED",
"message": "Cancelled booking cannot be paid"
}

2.Подтверждение платежа

Запрос:

Метод: POST
URL: /v1/payments/webhook
Заголовки:

Content-Type: application/json
X-Signature: <signature>

Ответ:

HTTP/1.1 200 OK
Content-Type: application/json
{
"paymentId": 8001,
"orderId": 2,
"paymentStatus": "paid",
"transactionId": "txn_123456",
"createdAt": "2024-05-01T12:30:00Z",
"updatedAt": "2024-05-01T12:31:05Z"
}

Ошибки:

HTTP/1.1 401 Unauthorized
Content-Type: application/json

{
"error": "INVALID_SIGNATURE",
"message": "Webhook signature verification failed"
}
HTTP/1.1 400 Bad Request
Content-Type: application/json
{
"error": "INVALID_PAYLOAD",
"message": "Webhook signature verification failed"
}

3.Отмена платежа

Метод: DELETE

URL: /v1/payments/{paymentId}

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTtoken>

Ответ:

HTTP/1.1 200 OK

Ошибки:

HTTP/1.1 400 Bad Request

Content-Type: application/json

{
"error": "PAYMENT_STATUS_PAID"
"message": "Paid payment cannot be canceled"
}

4.Получить статус платежа

Запрос:

Метод: GET

URL: /v1/payments/{paymentId}

Заголовки:

Authorization: Bearer <randomJWTToken>

Content-Type: application/json

Ответ:

HTTP/1.1 200 OK

Content-Type: application/json

{

"paymentStatus": "pending"

}

Ошибки:

HTTP/1.1 404 Not Found

Content-Type: application/json

{

"error": "NOT_FOUND",

"message": "Payment does not exist"

}

5.История платежей пользователя

Запрос:

Метод: GET

URL: /v1/payments?userId=11&page=1&limit=10

Заголовки:

Authorization: Bearer <randomJWTToken>

Content-Type: application/json

Ответ:

HTTP/1.1 200 OK

Content-Type: application/json

{

[

{

"paymentId": 234,

"bookingId": 22,

"userId": 11,

"paymentStatus": "pending",

"paymentMethod": "card",

"createdAt": 2026-02-28T14:30:00Z,

"amount": 1299.99,

"currency": RUB,

"paymentProvider": "Sberbank"

},

{

"paymentId": 343,

"bookingId": 12,

"userId": 11,

"paymentStatus": "paid",

"paymentMethod": "card",

"createdAt": 2026-02-25T14:30:00Z,

"amount": 1299.99,

"currency": RUB,

"paymentProvider": "Sberbank"

}

]

"pagination":

{

"page": 1

"limit": 10

"total": 2

"totalPages": 1

"hasNext": false

"hasPrev": false

}

}

/v1/bookings

1.Бронирование рабочего места

Запрос:

Метод: POST

URL: /v1/bookings/{placeId}

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTtoken>

{

"bookingStartTime": 2026-03-02T14:30:00Z,

"bookingEndTime": 2026-03-02T16:30:00Z

}

Ответ:

HTTP/1.1 201 Created

Content-Type: application/json

{
"bookingId": 2433,
"bookingStartTime": 2026-03-02T14:30:00,
"bookingEndTime": 2026-03-02T16:30:00Z,
"bookingStatus": "active",
"createdAt": 2026-02-28T14:25:00Z,
"updatedAt": 2026-02-28T14:25:02Z,
"paymentState": "notPaid"
}

Ошибки:

HTTP/1.1 400 Bad Request
Content-Type: application/json
{
"error": "BAD_REQUEST",
"message": "Wrong booking time format"
}

2. Отмена брони

Метод: PATCH

URL: /v1/bookings/{bookingId}

Заголовки:

Content-Type: application/json
Authorization: Bearer <randomJWTtoken>

{
"bookingStatus": "canceled"
}

Ответ:

HTTP/1.1 200 OK
Content-Type: application/json
{
"message": "Booking successfully canceled"
}

Ошибки:

HTTP/1.1 404 Not Found
Content-Type: application/json
{

"error": "NOT_FOUND",

"message": "Booking not found"

}

2.Получение информации о конкретной брони

Метод: GET

URL: /v1/bookings/{bookingId}

Заголовки:

Authorization: Bearer <randomJWTtoken>

Ответ:

HTTP/1.1 200 OK

Content-Type: application/json

{

"bookingId": 2433,

"bookingStartTime": 2026-03-02T14:30:00Z,

"bookingEndTime": 2026-03-02T16:30:00Z,

"bookingStatus": "active"

"createdAt": 2026-02-28T14:25:00Z,

"updatedAt": 2026-02-28T14:25:02Z,

"paymentState": "notPaid"

}

3.Получение списка броней пользователя

Метод: GET

URL: /v1/bookings?&page=1&limit=10

Заголовки:

Accept: application/json

Authorization: Bearer <randomJWTtoken>

Ответ:

HTTP/1.1

Content-Type: application/json

{

[

{

"bookingId": 2433,

"bookingStartTime": 2026-03-02T14:30:00Z,

"bookingEndTime": 2026-03-02T16:30:00Z,
"bookingStatus": "active"
"createdAt": 2026-02-28T14:25:00Z,
"updatedAt": 2026-02-28T14:25:02Z,
"paymentState": "notPaid"
},
{
"bookingId": 2434,
"bookingStartTime": 2026-02-25T14:30:00Z,
"bookingEndTime": 2026-02-25T16:30:00Z,
"bookingStatus": "expired",
"createdAt": 2026-02-23T14:30:00Z,
"updatedAt": 2026-02-23T14:30:02Z,
"paymentState": "Paid"
}
]
"pagination":
{
"page": 1,
"limit": 10,
"total": 2,
"totalPages": 1,
"hasNext": false,
"hasPrev": false
}
}

Ошибки:

HTTP/1.1 404 Not Found
Content-Type: application/json
{
"error": "NOT_FOUND",
"message": "Booking not found"
}

4.Получение списка активных броней пользователя:

Запрос:

Метод: GET

URL: /v1/bookings?bookingStatus=active&page=1&limit=10

Заголовки:

Accept: application/json

Authorization: Bearer <randomJWTtoken>

Ответ:

HTTP/1.1

Content-Type: application/json

{

[

{

"bookingId": 2433,

"bookingStartTime": 2026-03-02T14:30:00Z,

"bookingEndTime": 2026-03-02T16:30:00Z,

"bookingStatus": "active"

"createdAt": 2026-02-28T14:25:00Z,

"updatedAt": 2026-02-28T14:25:02Z,

"paymentState": "notPaid"

}

]

"pagination":

{

"page": 1,

"limit": 10,

"total": 1,

"totalPages": 1,

"hasNext": false,

"hasPrev": false,

}

}

Ошибки:

HTTP/1.1 404 Not Found

Content-Type: application/json

{

"error": "NOT_FOUND",

"message": "Booking not found"

}

/v1/reviews

1.Написание нового отзыва

Запрос:

Метод: POST

URL: /v1/reviews/{placeId}

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTtoken>

{

"rating": 4,

"comment": "All good, but location"

}

Ответ:

HTTP/1.1 201 Created

Content-Type: application/json

{

"reviewId": 243,

"userId": 11,

"rating": 4,

"comment": "All good, but location",

"createdAt": 2026-03-02T16:35:00Z

}

Ошибки:

HTTP/1.1 404 Not Found

Content-Type: application/json

{

"error": "NOT_FOUND",

"message": "Place does not exist"

}

2.Удаление отзыва пользователем

Метод: DELETE

URL: /v1/reviews/{reviewId}

Заголовки:

Authorization: Bearer <randomJWTtoken>

Ответ:

HTTP/1.1 200 OK

3.Получение отзывов о месте

Метод: GET

URL: /v1/places/{placeId}/reviews?page=1&limit=10

Ответ:


```
{
[
{
"userId": 11,
"rating": 4,
"comment": "All good, but location"
},
{
"userId": 13,
"rating": 5,
"comment": "All good"
}
],
"pagination":
{
"page": 1,
"limit": 10,
"total": 2,
"totalPages": 1,
"hasNext": false,
"hasPrev": false
}
}
```


Ошибки:

HTTP/1.1 404 Not Found

Content-Type: application/json


```
{
"error": "NOT_FOUND",
```

"message": "Place not found"

}

/v1/refresh

1.Обновление access токена

Запрос:

Метод: POST

URL: /v1/refresh

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTtoken>

Ответ:

HTTP/1.1 200 OK

Content-Type: application/json

{

"accessToken": <randomJWTToken2>

}

/v1/places

1.Получение информации о рабочем месте

Запрос:

Метод: GET

URL: /v1/places/{placeId}

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTtoken>

Ответ:

HTTP/1.1 200 OK

Content-Type: application/json

Cache-Control: public, max-age=300

ETag: "place-23-v5"

Last-Modified: 2026-03-02T16:35:00Z

{

"placeName": "WorkPlace1",

"type": "workTable",

"placeDescription": "Рабочее место за столом напротив окна с видом на улицу",

"placeLocation": "Russia, Moscow, Kiyevskaya St., 14",
"placeCapacity": 4,
"placeOccupancy": 0,
"ratin": 4.3,
"createdAt": 2025-03-02T16:35:00Z,
"updatedAt": 2026-02-28T16:35:00Z
}

Ошибки:

HTTP/1.1 404 Not Found
Content-Type: application/json
{
"error": "NOT_FOUND",
"message": "Place not found"
}

2.Обновление информации о рабочем месте

Метод: PATCH
URL: /v1/places/{placeId}
Заголовки:

Content-Type: application/json
Authorization: Bearer <randomJWTtoken>

{
"placeDescription":"Рабочее место без шумоподавления за столом напротив окна с видом на улицу с дорожным движением"
}

Ответ:
HTTP/1.1 200 OK

/v1/search

1.Поиск рабочего места

Запрос:

Метод:GET
URL: /v1/search?country=RU&city=Moscow&date=2022-03-01&page=1&limit=20

Ответ:

HTTP/1.1 200 OK
Content-Type: application/json

Cache-Control: public, max-age=300

ETag: "search-moscow-20260302-p1-v5"

Last-Modified: 2026-03-02T16:35:00Z

{

[

{

"placeId": 23,

"placeName": "WorkPlace1",

"placeLocation": "Russia, Moscow, Kiyevskaya St., 14",

"rating": 4.3,

"placeDescription": "Рабочее место за столом напротив окна с видом на улицу",

"placeOccupancy": 0

},

{

"placeId": 23,

"placeName": "WorkPlace2",

"placeLocation": "Russia, Moscow, Lenina St., 13",

"rating": 4.6,

"placeDescription": "Рабочее место за столом с ноутбуком в углу с шторкой",

"placeOccupancy": 0

"placeCapacity": 1

}

],

"pagination":

{

"page": 1,

"limit": 20,

"total": 2,

"totalPages": 1,

"hasNext": false,

"hasPrev": false

}

}

Ошибки:

HTTP/1.1 404 Not Found

Content-Type: application/json

{

"error": "NOT_FOUND",

"message": "Places with current query params was not found"

}

/v1/favorites

1.Добавить место в избранное

Запрос:

Метод: POST

URL: /v1/me/favorites

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTtoken>

{

"placeId": 22

}

Ответ:

HTTP/1.1 200 OK

Ошибки:

HTTP/1.1 409 Conflict

Content-Type: application/json

{

"error": "ALREADY_ADDED",

"message": "Place already in favorites"

}

HTTP/1.1 404 Not Found

Content-Type: application/json

{

"error": "NOT_FOUND",

"message": "Place not found"

}

2.Убрать место из избранного

Запрос:

Метод: DELETE

URL: /v1/me/favorites/{placeId}

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTtoken>

Ответ:

HTTP/1.1 200 OK

Ошибки:

HTTP/1.1 404 Not Found

Content-Type: application/json

{
"error": "NOT_FOUND",
"message": "Place not in favorites"
}

3.Получить список избранного

Запрос:

Метод: GET

URL: /v1/me/favorites?page=1&limit=10

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTtoken>

Ответ:

HTTP/1.1 200 OK

Content-Type: application/json

Cache-Control: public, max-age=60

ETag: "favorites-2-v3"

Last-Modified: 2026-03-02T16:35:00Z

{
"places":[
{
"placeId": 23,
"placeName": "WorkPlace1",
"placeLocation": "Russia, Moscow, Kiyevskaya St., 14",
"rating": 4.3,

"placeDescription": "Рабочее место за столом напротив окна с видом на улицу",
"placeOccupancy": 0
},
{
"placeId": 23,
"placeName": "WorkPlace2",
"placeLocation": "Russia, Moscow, Lenina St., 13",
"rating": 4.6,
"placeDescription": "Рабочее место за столом с ноутбуком в углу с шторкой",
"placeOccupancy": 0
"placeCapacity": 1
}
],
"pagination":
{
"page": 2,
"limit": 10,
"total": 2,
"totalPages": 1,
"hasNext": false,
"hasPrev": false
}
}

/v1/notifications

1. Информация об уведомлении

Запрос:

Метод: GET

URL: /v1/notification/{id}

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTtoken>

Ответ:

HTTP/1.1 200 OK

Content-Type: application/json

{
"type": "booking_confirmed",
"channel": "push",
"message": "Place successfully booked",
"status": "sent",
"createdAt": 2026-03-02T16:35:00Z,
"sentAt": 2026-03-02T16:35:30Z
}

Ошибки:

HTTP/1.1 404 Not Found

Content-Type: application/json

{
"error": "NOT_FOUND"
"message": "Notification not found"
}

2. Обновление настроек уведомлений

Запрос:

Метод: PATCH

URL: /v1/notifications/settings

Заголовки:

Authorization: Bearer <randomJWTToken>

Content-Type: application/json

{
  "emailNotifications": false
}

Ответ:

HTTP/1.1 200 OK

/v1/analytics

1.Получить логи рабочего места за определенную дату

Запрос:

Метод: GET

URL:/v1/analytics/{placeId}?from=2026-02-28T22:45:00Z&to=2026-02-28T23:00:00Z&page=1&limit=20

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTtoken>

Ответ:

HTTP/1.1 200 OK

Content-Type: application/json

{
"placeId": 23,
"from": "2026-02-28T22:45:00Z",
"to": "2026-02-28T23:00:00Z",
"events": [
{
"eventId": 98123,
"eventType": "booking_created",
"bookingId": 4432,
"userId": 112,
"timestamp": "2026-02-28T22:46:12Z"
},
{
"eventId": 98124,
"eventType": "payment_completed",
"bookingId": 4432,
"userId": 112,
"timestamp": "2026-02-28T22:47:03Z"
},
{
"eventId": 98125,
"eventType": "check_in",
"bookingId": 4432,
"userId": 112,
"timestamp": "2026-02-28T22:52:10Z"
},
{
"eventId": 98126,
"eventType": "check_out",
"bookingId": 4432,

"userId": 112,
"timestamp": "2026-02-28T22:59:48Z"
}
],
"pagination":
{
"page": 1,
"limit": 10,
"total": 4,
"totalPages": 1,
"hasNext": false,
"hasPrev": false
}
}

Ошибки:

HTTP/1.1 404 Not Found
Content-Type: application/json
{
"error": "NOT_FOUND"
"message": "No events at that time"
}

2.Добавление события в логи

Метод: POST

URL:/v1/analytics/{placeId}

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTtoken>

{
"eventType": "payment_completed",
"bookingId": 4432,
"userId": 112,
"timestamp": "2026-02-28T22:47:03Z"
}

Ответ:

HTTP/1.1 201 Created

Content-Type: application/json

{

"eventId": 99123,

"eventType": "payment_completed",

"bookingId": 4432,

"userId": 112,

"timestamp": "2026-03-01T13:47:03Z"

}

/v1/refunds

1.Создать заявку на возврат средств пользователю

Запрос:

Метод:POST

URL: /v1/refunds

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTtoken>

{

"paymentId": 3342,

"reason": "booking_canceling"

}

Ответ:

HTTP/1.1 201 Created

Content-Type: application/json

{

"refundId": 323,

"reason": "booking_canceling",

"status": "pending",

"createdAt": "2026-02-28T13:47:03Z"

}

Ошибка: refund in progress, please wait

HTTP/1.1 409 Conflict

Content-Type: application/json

{

"error": "REFUNG_IN_PROGRESS"

"message": "Refund already in progress, please wait"

2.Получение информации о возврате

Метод:GET

URL: /v1/refunds/{refundId}

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTToken>

Ответ:

HTTP/1.1

Content-Type: application/json

{
"refundId": 323,
"reason": "booking_canceling",
"status": "pending",
"createdAt": "2026-02-28T13:47:03Z",
"updatedAt": "2026-02-28T13:47:03Z"
}

Ошибки:

HTTP/1.1 404 Not Found

Content-Type: application/json

{
"error": "NOT_FOUND"
"message": "Refund not found"
}

3. Отмена возврата

Метод: DELETE

URL: /v1/refunds/{refundId}

Заголовки:

Content-Type: application/json

Authorization: Bearer <randomJWTToken>

Ответ:

HTTP/1.1 200 OK

Ошибки:

HTTP/1.1 404 Not Found

Content-Type: application/json

{

"error": "NOT_FOUND",
"message": "Refund not found"
}